Experiment No.9
Implement disk scheduling algorithms FCFS, SSTF.
Date of Performance: 24/03/2026
Date of Submission: 24/03/2026

Aim: To study and implement disk scheduling algorithms FCFS,SSTF.

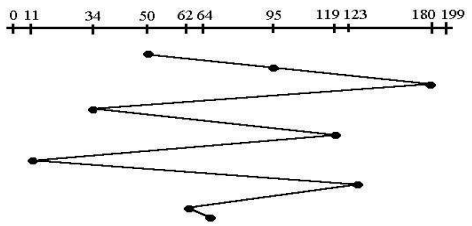
Objectives: The main purpose of disk scheduling algorithm is to select a disk request from the queue of IO requests and decide the schedule when this request will be processed.

Theory:
TYPES OF DISK SCHEDULING ALGORITHMS

Although there are other algorithms that reduce the seek time of all requests, I will only concentrate on the following disk scheduling algorithms:

1.First Come-First Serve (FCFS)
2.Shortest Seek Time First (SSTF)
3.Elevator (SCAN)
4.Circular SCAN (C-SCAN)
5.C-LOOK

Given the following queue -- 95, 180, 34, 119, 11, 123, 62, 64 with the Read-write head initially at the track 50 and the tail track being at 199 let us now discuss the different algorithms.

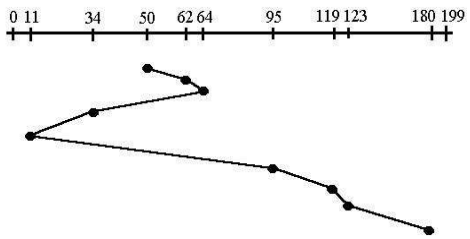


FCFS

1. First Come -First Serve (FCFS)

All incoming requests are placed at the end of the queue. Whatever number that is next in the queue will be the next number served. Using this algorithm doesn't provide the best results. To determine the number of head movements you would simply find the number of tracks it took to move from one request to the next. For this case it went from 50 to 95 to 180 and so on. From 50 to 95 it moved 45 tracks. If you tally up the total number of tracks you will find how many tracks it had to go through before finishing the entire request. In this example, it had a total head movement of 640 tracks. The disadvantage of this algorithm is noted by the

oscillation from track 50 to track 180 and then back to track 11 to 123 then to 64. As you will soon see, this is the worse algorithm that one can use.



SSTF

2.Shortest Seek Time First (SSTF)

In this case request is serviced according to next shortest distance. Starting at 50, the next shortest distance would be 62 instead of 34 since it is only 12 tracks away from 62 and 16 tracks away from 34. The process would continue until all the process are taken care of. For example the next case would be to move from 62 to 64 instead of 34 since there are only 2 tracks between them and not 18 if it were to go the other way. Although this seems to be a better service being that it moved a total of 236 tracks, this is not an optimal one. There is a great chance that starvation would take place. The reason for this is if there were a lot of requests close to eachother the other requests will never be handled since the distance will always be greater.

Implementation:

Code:

```
#include <stdio.h>
#include <stdlib.h>

void FCFS(int requests[], int n, int head) {
    int totalSeekTime = 0;

    printf("\n--- FCFS Disk Scheduling ---\n");
    printf("Seek Sequence: %d", head);
```

```
    for (int i = 0; i < n; i++) {
        totalSeekTime += abs(requests[i] - head);
        head = requests[i];
        printf(" -> %d", head);
    }

    printf("\nTotal Seek Time (FCFS): %d\n", totalSeekTime);
}

void SSTF(int requests[], int n, int head) {
    int visited[100] = {0};
    int totalSeekTime = 0;

    printf("\n--- SSTF Disk Scheduling ---\n");
    printf("Seek Sequence: %d", head);

    for (int count = 0; count < n; count++) {
        int minDist = 9999, index = -1;

        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                int dist = abs(requests[i] - head);
                if (dist < minDist) {
                    minDist = dist;
                    index = i;
                }
            }
        }

        visited[index] = 1;
        totalSeekTime += minDist;
        head = requests[index];
        printf(" -> %d", head);
    }

    printf("\nTotal Seek Time (SSTF): %d\n", totalSeekTime);
}

int main() {
    int requests[100], n, head;

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter the request sequence:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &requests[i]);
    }

    printf("Enter initial head position: ");
```

```
    scanf("%d", &head);

    // Run BOTH algorithms
    FCFS(requests, n, head);
    SSTF(requests, n, head);

    return 0;
}
```

Output:

```
PS C:\Users\student\Desktop\pr\schedulin algo> cd "c:\Users\student\Desktop\pr\schedulin algo\" ; if ($?) { gcc disk.c -o disk } ; if ($?) { .\disk }
Enter number of requests: 6
Enter the request sequence:
123
874
692
475
105
376
Enter initial head position: 345

--- FCFS Disk Scheduling ---
Seek Sequence: 345 -> 123 -> 874 -> 692 -> 475 -> 105 -> 376
Total Seek Time (FCFS): 2013

--- SSTF Disk Scheduling ---
Seek Sequence: 345 -> 376 -> 475 -> 692 -> 874 -> 123 -> 105
Total Seek Time (SSTF): 1298
```

Conclusion:

Disk scheduling algorithms determine the order in which disk I/O requests are serviced, directly affecting system performance, seek time, and response time. Different algorithms are designed with varying priorities such as fairness, efficiency, and simplicity.

FCFS (First-Come, First-Served) is the simplest approach, servicing requests in arrival order. While fair, it often results in high seek time and poor performance due to unnecessary disk head movement.

SSTF (Shortest Seek Time First) selects the request closest to the current head position, reducing total seek time. It improves efficiency but may cause starvation for requests far from the current head position.

SCAN (Elevator Algorithm) moves the disk head in one direction, servicing requests along the way, then reverses direction. It provides better performance and more uniform wait times compared to FCFS and SSTF.

C-SCAN (Circular SCAN) services requests in one direction only and quickly returns to the beginning without servicing requests on the return trip. This ensures more uniform waiting time than SCAN.

LOOK and C-LOOK are improvements over SCAN and C-SCAN, respectively. They only move the disk head as far as the last request in each direction instead of going to the physical end of the disk, reducing unnecessary movement and improving efficiency.